Distribution Module

> ***Order to outlet, one trail.*** *Brand → Distributor → FE/Salesman → Outlet → Consumer*

A complete supply-chain & distribution module for Kinematic, spanning the backend API, web dashboard, Android FE app, and iOS FE app. Designed for Indian FMCG distribution — full GST invoicing (CGST/SGST/IGST, HSN, e-invoice IRN, e-way bill), a deterministic scheme engine, double-entry ledger with credit-limit enforcement, and offline-first order capture for field staff.

Table of contents

1. Repos & ownership
2. The five-step flow
3. Architecture at a glance
4. Database schema
5. API surface
6. Scheme engine
7. Mobile order-capture flow
8. Dashboard pages
9. Anti-fraud / integrity controls
10. Operational runbook
11. Testing & verification
12. Open items / future work

Repos & ownership

Repo	Role	Tech	Branch (in this rollout)
`Shaggywize63/Kinematic`	Backend API	Express + TypeScript + Supabase + Zod	`claude/supply-chain-module-P5biq`
`Shaggywize63/kinematic-dashboard`	Admin dashboard	Next.js 14 (App Router) + Tailwind	`claude/supply-chain-module-P5biq`
`Shaggywize63/Kinematic-App`	Android FE app	Kotlin + Jetpack Compose + Hilt + Retrofit + Room + WorkManager	`claude/supply-chain-module-P5biq`
`Shaggywize63/Kinematic-iOS`	iOS FE app	Swift + SwiftUI + URLSession + Combine	`claude/supply-chain-module-P5biq`

The four repos share one Supabase project (`lnvxqjqfsxvtjvbzphou`) and one Railway-hosted Express API.

The five-step flow

The deck slide **04 · Distribution** defines five steps. Every table, route, and screen below maps to one of them.

#	Step	Activities	Persona
1	**Brand**	Plan, targets, prices	Brand admin (dashboard)
2	**Distributor**	Stock, billing, schemes	Distributor / admin (dashboard)
3	**FE / Salesman**	Order capture, payment, returns	Field executive (mobile)
4	**Outlet**	Delivered, billed, audited	Driver/dispatcher + outlet (POD on mobile, audit on dashboard)
5	**Consumer**	On-shelf, in-hand	FE captures via planograms + secondary-sales

Architecture at a glance

```
┌──────────────────────────────────────────────────────────────────────┐
│                  Kinematic — Distribution Module                     │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  Dashboard (Next.js)         Mobile (Android · iOS)                  │
│       │                              │                               │
│       │  REST                        │  REST + Idempotency-Key       │
│       ▼                              ▼                                │
│  ┌──────────────────────────────────────────────────────┐            │
│  │           Express API  (Kinematic backend)           │            │
│  │  /api/v1/distribution/*    /api/v1/salesman/*        │            │
│  │  • idempotency middleware  • upload-signer           │            │
│  │  • order-pricer  • scheme-engine  • tax  • einvoice  │            │
│  └──────────────────────────────────────────────────────┘            │
│                            │                                         │
│                            ▼                                         │
│  ┌──────────────────────────────────────────────────────┐            │
│  │          Supabase · Postgres + Storage               │            │
│  │  24 distribution tables  +  audit_log  +  ledger     │            │
│  │  RLS · double-entry trigger · advisory locks         │            │
│  └──────────────────────────────────────────────────────┘            │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

Module integration points

```
users, stores (...), skus, route_plans, route_plan_outlets,
visit_logs, cities, zones, planograms, user_module_permissions, modules
```

- **Reuses existing middleware:** `requireAuth`, `requireModule(...)`, `enforceCityScope`, demo-token bypass (`isDemo`)
- **Adds new modules to the permissions table:** 11 keys (see Permissions)

Database schema

24 new tables across 8 migrations. All tables carry `org_id uuid not null` + `client_id uuid null` for multi-tenant isolation. Migrations are idempotent (`CREATE TABLE IF NOT EXISTS`) and live as raw SQL files in the backend repo root, applied via the existing `src/scripts/run_migration.ts` runner or through the Supabase MCP `apply_migration` tool.

Migrations (in order)

File	Tables
`migration_distribution_master.sql`	`brands`, `distributors`, `distributor_users`, `outlet_distribution_ext`, `salesman_ext`, `product_distribution_ext`, `price_lists`, `price_list_items`
`migration_distribution_audit_idempotency.sql`	`audit_log`, `idempotency_keys`
`migration_distribution_orders.sql`	`orders`, `order_items` (+ `gen_order_no`)
`migration_distribution_invoicing.sql`	`invoices`, `invoice_items`, `dispatches`, `dispatch_lines`, `deliveries` (+ `gen_invoice_no`, `gen_dispatch_no`)
`migration_distribution_payments_returns.sql`	`payments`, `returns`, `return_items`, `ledger_entries` (+ `enforce_no_negative_balance` trigger, `post_ledger_entry` helper, `gen_payment_no`, `gen_return_no`)
`migration_distribution_schemes.sql`	`schemes`, `scheme_application_log`
`migration_distribution_consumer.sql`	`secondary_sales`
`migration_distribution_ledger_ordering_fix.sql`	(Patch) `clock_timestamp()` default + `(id DESC)` tiebreaker on `post_ledger_entry`

Master data

```
brands                      — legal brand identity (GSTIN, state, logo)
distributors                — super_stockist | distributor | wholesaler
distributor_users           — links distributor staff (in users) to a distributor
```

```
outlet_distribution_ext      — 1:1 with stores. GSTIN, customer_class, credit_limit, geofence
salesman_ext                 — 1:1 with users. daily_order_cap, daily_collection_cap, return_
product_distribution_ext     — 1:1 with skus. HSN, gst_rate, cess_rate, MRP, return_window_da
price_lists                  — versioned by (customer_class, region)
price_list_items             — per-SKU base price under a list
```

Transactions

```
orders                       — captured by FE/dashboard. status: placed → approved → invoiced
                               (or cancelled). Pins price_list_id + price_list_version.
                               Carries gps + geofence_passed + idempotency_key.
order_items                  — line items with sku snapshot, taxes, scheme_id+version

invoices                     — issued from approved orders. Carries IRN, eway_bill_no,
                               dispatch_id. UNIQUE partial index prevents double-issue
                               per order.
invoice_items                — IMMUTABLE snapshot of order_items at issue time.

dispatches                   — vehicle-level grouping. status: prepared → out → delivered
dispatch_lines               — many-to-many between dispatches and invoices.
deliveries                   — POD (image + signature + GPS), one per invoice.

payments                     — outlet payments. mode: cash | upi | cheque | credit_adjustment
                               CHECK constraint: cheque ⇒ cheque_image_url required.
                               status: pending (cheque) | cleared | bounced | cancelled.

returns                      — outlet returns. photo_urls[] (≥1 enforced via CHECK).
                               status: requested → supervisor_approved → credited (or rejecte
                               requires_supervisor flag from salesman_ext threshold.
return_items                 — line snapshot of original invoice items.
```

Schemes

```
schemes                      — versioned. type: QPS | SLAB_DISCOUNT | BXGY | VALUE_DISCOUNT
                               targeting jsonb (customer_class, sku_ids, outlet_ids, ...)
                               rules     jsonb (per-type config)
                               priority + stackable + valid_from/to.
                               Editing inserts a new version row.
scheme_application_log       — append-only proof. (order_id, scheme_id, scheme_version,
                               engine_version, inputs jsonb, outputs jsonb).
```

Ledger (double-entry)

```
ledger_entries
  outlet_id, distributor_id
  entry_type: invoice | payment | return | credit_note | adjustment
  ref_table, ref_id           — back-reference to source row
  dr, cr                      — CHECK ((dr=0 OR cr=0) AND (dr+cr>0))
  running_balance             — computed by post_ledger_entry helper
  posted_at = clock_timestamp()  — distinct even within one transaction
  posted_by, posted_by_role   — role-aware override hooks
```

```
trigger trg_ledger_no_negative
  refuses an insert that pushes running_balance > credit_limit
  UNLESS posted_by_role IN ('super_admin','admin','main_admin')
  → admin override is itself a ledger row + audit_log entry.

function post_ledger_entry(...)
  - takes pg_advisory_xact_lock(hashtext(outlet_id::text)) so concurrent
    posts for the same outlet serialise.
  - reads previous balance ORDER BY posted_at DESC, id DESC LIMIT 1.
  - writes the new row with clock_timestamp() and updates the
    outlet_distribution_ext.current_balance mirror atomically.
```

Cross-cutting

```
audit_log                    — append-only. RLS denies UPDATE/DELETE.
                               (actor_user_id, actor_role, action, entity_table,
                                entity_id, before jsonb, after jsonb, metadata,
                                ip_address, user_agent)
idempotency_keys             — short-lived replay cache (24h TTL).
                               key (PK), org_id, user_id, route, request_hash,
                               response_status, response_body jsonb.
secondary_sales              — outlet × SKU × period × source. UNIQUE constraint
                               prevents duplicate captures per (org, outlet, sku,
                               period_start, period_end, source).
```

Permissions

The 11 module keys registered in the `modules` table and granted to each non-super-admin user via `user_module_permissions` (super_admin always has access via the existing `requireModule` bypass):

```
distribution                  Overview KPIs
distribution_brands           Brands CRUD
distribution_pricing          Price lists CRUD + activation
distribution_schemes          Schemes editor + preview
distribution_distributors     Distributors CRUD
distribution_orders           Orders list + detail + approve + cancel
distribution_invoicing        Invoices + dispatches
distribution_payments         Payment register
distribution_returns          Returns approval queue
distribution_ledger           Outlet ledger + ageing
distribution_consumer         Secondary-sales capture
```

API surface

All routes prefixed `/api/v1/`. Mounted in `src/app.ts`. Every mutation accepts an `Idempotency-Key` header — middleware in `src/middleware/idempotency.ts` short-circuits replays via `idempotency_keys` (24h TTL). Pagination uses `?page=&limit=`.

Brand step (admin only)

Method	Path	Notes
`GET / POST / PATCH / DELETE`	`/distribution/brands[/:id]`	requireAdminOrAbove on writes
`GET / POST`	`/distribution/price-lists[/:id]`	new list = new auto-bumped version, starts inactive
`POST`	`/distribution/price-lists/:id/items:bulk`	refuses if list is active
`POST`	`/distribution/price-lists/:id/activate`	atomically deactivates the previous active list for the same (class, region)
`GET / POST / POST`	`/distribution/schemes[/:id][/deactivate]`	edit = new version row, older deactivated
`POST`	`/distribution/schemes/preview`	dry-run a cart to see schemes applied

Distributor step

Method	Path	Notes
`GET / POST / PATCH / DELETE`	`/distribution/distributors[/:id]`	requireAdminOrAbove on writes
`GET`	`/distribution/distributors/:id/billing-summary`	open orders + dispatched + invoiced + ageing buckets

FE / Salesman step (mobile-facing)

All under `/api/v1/salesman/*`. Auth + `enforceCityScope`.

Method	Path	Notes
`GET`	`/salesman/route/today`	beat plan + outstanding balance + last-order suggestion per outlet
`POST`	`/salesman/visits/:visitId/checkin`	haversine vs outlet `geofence_radius_m`
`GET`	`/salesman/outlets/:id/cart-suggest`	last 3 orders + reorder reco
`POST`	`/salesman/orders/preview`	runs `priceCart` + `applySchemes`. No persist.
`POST`	`/salesman/orders`	persists order + items + scheme_application_log. Idempotent.
`GET`	`/salesman/orders[?status=]`	salesman's own orders

GET	/salesman/orders/:id	detail
POST	/salesman/orders/:id/cancel	only own orders if FE; only placed or approved
POST	/salesman/payments	cash/UPI/cheque/credit-adj. Cheque blocks without our-issued image URL. Idempotent.
POST	/salesman/returns	photo URLs validated, supervisor flag derived from threshold. Idempotent.
POST	/salesman/secondary-sales	off-take capture
POST	/salesman/uploads/sign	5-min Supabase Storage signed PUT URL, bucket-scoped path

Outlet step (back-office)

Method	Path	Notes
GET / POST	`/distribution/orders[/:id]/[approve	cancel]`
GET / POST	/distribution/invoices[/:id]/[cancel]	issue from approved order; cancel writes credit-note ledger reversal
GET / POST / POST / POST	`/distribution/dispatches[/:id]/[eway-bill	mark-out]`
POST	/distribution/deliveries	POD URL must come from signed-uploads
GET / POST / POST	/distribution/payments[/:id/status]	cheque clearance (cleared
GET / POST / POST	`/distribution/returns[/:id/[approve	reject]]`
GET	/distribution/ledger[?outlet_id=&distributor_id=]	entries view
GET	/distribution/ledger/ageing	total + 0-30 / 31-60 / 61-90 / 90+ buckets

Consumer step

Method	Path	Notes
GET / POST	/distribution/secondary-sales	dashboard side
POST	/salesman/secondary-sales	mobile capture
(reused)	/api/v1/planograms/*	existing module — joins for compliance

Scheme engine

Lives at `src/services/scheme-engine.ts`. **Pure, deterministic, server-authoritative.** Pinned engine version: `scheme-engine-1.0.0`.

Inputs

- Output of `order-pricer.priceCart(...)` (cart with HSN, GST, MRP)
- Customer context (org, customer_class, outlet_id, intra_state, date)

Pipeline

1. Fetch active schemes overlapping `date` ordered by `priority asc`
2. For each scheme:
    - Skip if `targeting` doesn't match (customer_class / outlet_ids / sku_ids)
    - Apply by type (see below)
    - If non-stackable, mark per-SKU "consumed"
3. Re-summarise totals (CGST/SGST/IGST/cess) after discounts; banker's rounding
4. Return `{ lines, applied, scheme_total, discount_total }`

Types

Type	Rules shape	Effect
`QPS`	`{ target_sku_id, slabs: [{min_qty, free_qty, free_sku_id?}] }`	Picks highest matching slab on `qty` of target SKU; appends a free-good line
`SLAB_DISCOUNT`	`{ sku_ids:[], slabs:[{min_qty, percent}] }`	Highest matching slab → % discount on each matching line; tax recomputed on discounted base
`BXGY`	`{ buy_sku, buy_qty, get_sku, get_qty, max_per_order }`	Inserts free-good line `unit_price=0, is_free_good=true`
`VALUE_DISCOUNT`	`{ min_value, percent? OR flat_amount? }`	Order-level cut allocated proportionally across non-free lines

Anti-tamper guarantee

Client previews (online preview endpoint, or local cache for offline) but the server **always re-runs the engine** at order create AND at invoice issue.

- Persisted price/tax = server's only.
- If client sends `client_total` and it differs from server total by >₹0.01 → `409 PRICE_MISMATCH` with `{server_total, client_total}`. Client must re-preview and re-confirm before resubmit.
- Each applied scheme writes to `scheme_application_log` with `engine_version`, inputs, outputs —

Mobile order-capture flow

Target: order capture in **≤3 taps** from outlet check-in. Offline-first, GPS-stamped, deterministic.

```
RoutePlan ──► Outlet detail ──► OrderCart ──► OrderReview ──► (queued/synced)
   1 tap            2 taps          3 taps         confirm
```

Sequence

1. **RoutePlan** (existing screen) — tap outlet card
2. **Outlet detail** (existing `LogVisitScreen` / `StoreVisitView`) — "Start Order" CTA. Geofence checked client-side from cached `outlet.lat/lng/radius`; outside the fence → soft-block with override that flags `geofence_passed=false` server-side
3. **OrderCart** — pre-populated from `cart-suggest` (last 3 orders + planogram reco). +/- chips per SKU. "Apply Schemes" → online preview; offline shows last-known prices with "preview pending" badge
4. **OrderReview** — server-priced cart + applied schemes + GST split
5. **Confirm** persists locally with stable `Idempotency-Key` (UUID at confirm time). Sync worker posts with exponential backoff; on `409 PRICE_MISMATCH` re-preview, re-confirm
6. (Optional) **PaymentCollect** — cash/UPI/cheque against open invoices; cheque path forces camera capture
7. (Optional) **Returns** — from outlet detail; ≥1 photo enforced

Offline queue contract

- Queue scoped by **user-key** (last 24 chars of access token) so re-login under a different account never flushes the previous user's writes.
- Idempotency-Key generated at confirm time and persisted with the row, so retries reuse the same key — server returns the original `order_no` on replay.
- 4xx (validation / price-mismatch) → keep the row + `lastError` so the UI can surface it. 5xx / network → exponential backoff via WorkManager (Android) or reachability listener (iOS).

Android (Kinematic-App)

Concern	File
Routes	`app/src/main/java/com/kinematic/app/ui/navigation/NavGraph.kt` (sealed `Route` adds `OrderCart`, `OrderReview`, `OrderHistory`, `OrderDetail`, `PaymentCollect`, `Returns`)
Room entities	`app/src/main/java/com/kinematic/app/data/local/OfflineOrder.kt` (`OfflineOrder`, `OfflinePayment`, `OfflineReturn`)
DAO	`app/src/main/java/com/kinematic/app/data/local/DistributionDao.kt`

DB version	`app/src/main/java/com/kinematic/app/data/local/KinematicDatabase.kt` (v6)
Repository	`app/src/main/java/com/kinematic/app/data/repository/DistributionRepository.kt`
Sync worker	`app/src/main/java/com/kinematic/app/workers/DistributionSyncWorker.kt` (HiltWorker)
API	`app/src/main/java/com/kinematic/app/data/api/KinematicApi.kt` (`distSubmitOrder` etc., with `@Header("Idempotency-Key")`)
ViewModel	`app/src/main/java/com/kinematic/app/viewmodel/DistributionViewModel.kt`
Screens	`app/src/main/java/com/kinematic/app/ui/screens/DistributionScreens.kt`, `PaymentCollectScreen.kt`, `ReturnScreen.kt`

iOS (Kinematic-iOS)

Concern	File
Routes	`Kinematic/Kinematic/KinematicApp.swift` (`enum SecondaryRoute` adds `orderHistory`, `orderCart`, `orderReview`, `orderDetail`, `paymentCollect`, `returns`, `distributorStock`, `secondarySales`)
Models	`Kinematic/Kinematic/Models/DistributionModels.swift`
Disk-persisted queue	`Kinematic/Kinematic/Services/OrderCache.swift` (atomic JSON writes, scoped by user-key)
API client	`Kinematic/Kinematic/Services/DistributionAPI.swift` (async/await + Idempotency-Key)
ViewModel	`Kinematic/Kinematic/ViewModels/DistributionViewModel.swift`
Views	`Kinematic/Kinematic/Views/Distribution/{OrderCartView, OrderReviewView, OrderHistoryView, OrderDetailView, PaymentCollectView, ReturnView, ComingSoonView}.swift`
Route host	`Kinematic/Kinematic/Views/Navigation/SecondaryScreenHost.swift`

Dashboard pages

All under `src/app/dashboard/distribution/` in the kinematic-dashboard repo. Sidebar group **"Distribution"** added in `src/app/dashboard/layout.tsx`.

Path	Purpose	Module key
`/dashboard/distribution`	Overview KPIs (GMV today, pending approval, recent orders)	`distribution`

`.../brands`	Brands list + create form	`distribution_brands`
`.../distributors`	Distributors directory + create	`distribution_distributors`
`.../price-lists`	Versioned lists, activate flow	`distribution_pricing`
`.../schemes`	Type-aware editor (QPS / SLAB / BxGy / VALUE), JSON targeting + rules	`distribution_schemes`
`.../orders` + `/[id]`	List + filter + detail with GST split + approve/cancel	`distribution_orders`
`.../invoices`	List + IRN status + EWB indicator + issue + cancel	`distribution_invoicing`
`.../dispatches`	EWB attach + mark-out (gated by ₹50k threshold in UI + server)	`distribution_invoicing`
`.../payments`	Filter by mode + status; cheque/UPI status pills	`distribution_payments`
`.../returns`	Supervisor approval queue with photo count + reason pill	`distribution_returns`
`.../ledger`	Ageing buckets, entries view with DR/CR coloring + outlet filter	`distribution_ledger`
`.../secondary-sales`	Off-take capture/list + planogram-link reference	`distribution_consumer`

Style follows existing dashboard patterns — inline tables (no DataTable primitive), CSS variables (`--primary` is brand red `#E01E2C`), reuse `ConfirmModal`, `ClientSelect`, `CitySelect`, `StoreSelect`. New shared atoms in `src/components/distribution/Atoms.tsx` (`StatCard`, `Pill`, `Btn`, `Th/Td`, `inr`, `statusColor`).

Anti-fraud / integrity controls

Server-side guards. None can be disabled by the client.

1. **Idempotency keys** — middleware on every distribution mutation. Same key + same body → cached response; different body → 409. 24h TTL.
2. **Server-authoritative pricing** — scheme engine runs on order create AND invoice issue; client total mismatch >₹0.01 → 409 `PRICE_MISMATCH`.
3. **Price-list version pinning** — `orders.price_list_version` written from the server's view; mid-flight version change → 409 `PRICE_VERSION_CHANGED`. Historical orders display priced with their pinned version forever.
4. **Double-entry ledger** + `enforce_no_negative_balance` trigger (admin role override is itself a ledger row + audit_log entry).
5. **GPS + geofence** — order/payment/return require `gps`; haversine vs outlet `geofence_radius_m`. Failed geofence flagged `geofence_passed=false` and surfaced in the supervisor queue.
6. **Role-based monetary caps** — `salesman_ext.daily_order_cap_value`, `daily_collection_cap`, `single_order_cap_value` enforced cumulatively per day; overflow → 403.
7. **Cheque integrity** — `payments` insert with `mode=cheque` rejects without `cheque_image_url` (DB CHECK

8. **Returns control** — refuses without ≥1 photo (CHECK), valid `reason_code`, `original_invoice_id` within `return_window_days`. Above `salesman_ext.return_threshold_value` → `requires_supervisor=true`.
9. **e-Way bill enforcement** — invoice `grand_total > 50000` blocks `dispatch.mark-out` until `eway_bill_no` present.
10. **Immutable audit_log** via `src/utils/audit.ts` helper; RLS revokes UPDATE/DELETE.
11. **Signed uploads only** — `/uploads/sign` issues 5-min Supabase Storage PUT URLs with bucket-scoped path `org/{org_id}/distribution/{kind}/{uuid}.jpg`; controllers verify URLs match this prefix before accepting them on cheque / POD / return rows.
12. **Versioned schemes** — editing inserts a new `version` row; older versions deactivated atomically. `scheme_application_log` records each application with `engine_version`, inputs, outputs.
13. **Demo bypass parity** — `isDemo(user)` returns canned fixtures across all new endpoints (matches existing convention) for safe walkthroughs.

Operational runbook

1. Apply migrations (ordered)

```
-- via Supabase MCP apply_migration or src/scripts/run_migration.ts
1. migration_distribution_master.sql
2. migration_distribution_audit_idempotency.sql
3. migration_distribution_orders.sql
4. migration_distribution_invoicing.sql
5. migration_distribution_payments_returns.sql
6. migration_distribution_schemes.sql
7. migration_distribution_consumer.sql
8. migration_distribution_ledger_ordering_fix.sql
```

2. Register the modules + grant permissions

```
INSERT INTO public.modules (id, name, description) VALUES
  ('distribution',               'Distribution',          'Order to outlet, one trail.'),
  ('distribution_brands',        'Distribution · Brands', 'Brand identities (GSTIN, place-of-
  ('distribution_pricing',       'Distribution · Pricing','Versioned price lists per custome
  ('distribution_schemes',       'Distribution · Schemes','QPS / slab / BxGy / value-discoun
  ('distribution_distributors',  'Distribution · Distributors', 'Distributors / super-stockis
  ('distribution_orders',        'Distribution · Orders', 'FE / dashboard captured orders, a
  ('distribution_invoicing',     'Distribution · Invoicing', 'Invoices + dispatches + e-way b
  ('distribution_payments',      'Distribution · Payments', 'Cash / UPI / cheque / credit-ad
  ('distribution_returns',       'Distribution · Returns','Photo + reason + supervisor-gated
  ('distribution_ledger',        'Distribution · Ledger', 'Double-entry outlet ledger and age
  ('distribution_consumer',      'Distribution · Consumer', 'Secondary sales / planogram-lin
ON CONFLICT (id) DO NOTHING;

INSERT INTO public.user_module_permissions (user_id, module_id)
SELECT u.id, p.module_id
FROM public.users u
```

```
CROSS JOIN (VALUES
  ('distribution'),('distribution_brands'),('distribution_pricing'),
  ('distribution_schemes'),('distribution_distributors'),('distribution_orders'),
  ('distribution_invoicing'),('distribution_payments'),('distribution_returns'),
  ('distribution_ledger'),('distribution_consumer')
) AS p(module_id)
WHERE u.role IN ('super_admin','admin','sub_admin','client','city_manager','supervisor')
  AND u.is_active = true
ON CONFLICT DO NOTHING;
```

3. Seed minimum demo data per org

For a working demo, an org needs at least: one brand, one distributor, ≥1 SKUs with `product_distribution_ext` rows, an active price list with items, an `outlet_distribution_ext` row per target outlet, and a `salesman_ext` row per FE.

4. Deploy order

1. **Backend** (Railway) — merge & redeploy first so `/api/v1/distribution/*` exists
2. **Dashboard** (Vercel) — merge & redeploy second; UI hits the backend
3. **Android** (Play track) — merge & cut a release
4. **iOS** (TestFlight) — merge & cut a release

5. Storage bucket

Create a Supabase Storage bucket named `distribution` (or set the `SUPABASE_STORAGE_BUCKET` env var). Used by `/uploads/sign` for cheque, POD, return, signature, and KYC images.

6. Environment variables

Var	Where	Purpose
`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`	backend	already required; used by upload-signer's URL validator
`SUPABASE_STORAGE_BUCKET`	backend	optional; default `distribution`
`EINVOICE_GSP_URL`, `EINVOICE_GSP_USER`	backend	when present, switches IRN generation to live GSP (currently stub)
`EWAY_GSP_URL`, `EWAY_GSP_USER`	backend	same for e-Way bill

Testing & verification

No test framework is wired into the repos. The verification harness has three layers.

Layer 1 — Backend smoke (curl / Postman / SQL)

```
-- order → invoice → ledger DR → payment → ledger CR
-- (see migration_distribution_*.sql for the actual stored procs used)
```

Already validated on staging in this rollout:

- ✓ Order to invoice happy path (intra-state CGST+SGST split)
- ✓ Idempotency-Key replay → unique_violation
- ✓ Cheque without image → check_violation
- ✓ FE post over credit-limit → trigger raises check_violation
- ✓ Admin override path → succeeds (audit-trailed)
- ✓ Same-txn ledger ordering → fixed via clock_timestamp()

Layer 2 — Dashboard smoke

- Each new page renders without console errors as `super_admin`, `admin`, `supervisor`, `executive` (last sees mostly 403 cards)
- Brands + price-lists + schemes create flows
- Orders list filters work; detail shows GST split, applied schemes, audit
- Invoice cancel writes a credit-note ledger entry
- Returns approval queue → supervisor approve → ledger reflects credit
- Demo token: every page renders mock data via `isDemo(user)` bypass

Layer 3 — Mobile manual cases

- Login as FE; route-plan loads; tap outlet; geofence check
- Create order with 5 items in airplane mode → confirm → "Pending sync" banner → re-enable network → exactly one server-side row (idempotent)
- Force a 409 by editing the price-list mid-flight → verify app re-previews and shows server numbers before retry
- Cheque payment: camera path forced; submit blocked until image present
- POD: photo + signature at delivery; offline POD syncs on reconnect
- Return >₹threshold → "Awaiting supervisor approval" → admin clears on dashboard → FE sees status change
- Crash/kill mid-order → relaunch → draft survives → can resume
- Switch FE user → previous user's pending queue does NOT flush

---

Open items / future work

Item	Owner	Notes
GSP vendor for live IRN + e-way bill	Business / IT	Currently stub. NIC sandbox vs ClearTax / Cygnet — config switch per `client_id`.

UPI gateway webhook	Business / Backend	Razorpay vs Pioneer e; schema already has `gateway_payload jsonb`. ~1 week add-on.
Scheme stacking semantics	PM / Brand team	`priority asc + stackable bool` defaults to non-stackable per SKU. Confirm legal/commercial rules.
Free-goods GST treatment	Legal	Per-scheme `tax_on_mrp` flag exists in `rules` JSON; sign-off needed per brand.
Returns time limits	PM	`product_distribution_ext.return_window_days` (default 30). Confirm per-client default.
Distributor-to-distributor transfers	v2	Out of scope for now.
Multi-currency	v2	All `payments.currency` defaults to INR.
Monthly balance snapshots	Backend	If `ledger_entries` grows large; M2.5 add-on.
CI / test framework	Backend	None today. Postman + Newman as a tripwire is the cheapest M2.5 step.
Distributor stock screen (mobile)	M2.5	Wireframe ready; deferred.

Appendix — key files at a glance

```
Kinematic (backend)
├── migration_distribution_*.sql                    (8 files)
├── src/middleware/idempotency.ts                   request-replay protection
├── src/utils/audit.ts                              immutable trail helper
├── src/utils/upload-signer.ts                      signed-URL issuance + validator
├── src/services/order-pricer.ts                    deterministic price resolution
├── src/services/scheme-engine.ts                   QPS/SLAB/BXGY/VALUE engine
├── src/services/tax.ts                             CGST+SGST/IGST split
├── src/services/einvoice.ts                        IRN stub (GSP swap-ready)
├── src/services/eway-bill.ts                       EWB stub + threshold helper
├── src/controllers/distribution/                   (12 controllers)
└── src/routes/distribution/                        (12 routers)

kinematic-dashboard
├── src/app/dashboard/layout.tsx                    sidebar nav group
├── src/app/dashboard/distribution/                 (12 pages)
├── src/components/distribution/Atoms.tsx           shared atoms
└── src/lib/api.ts                                  ApiClient additions

Kinematic-App (Android)
├── app/.../data/local/OfflineOrder.kt              Room entities
├── app/.../data/local/DistributionDao.kt
├── app/.../data/repository/DistributionRepository.kt
├── app/.../data/api/KinematicApi.kt                Retrofit additions
```

```
  ├── app/.../workers/DistributionSyncWorker.kt
  ├── app/.../viewmodel/DistributionViewModel.kt
  ├── app/.../ui/navigation/NavGraph.kt               Route additions
  └── app/.../ui/screens/{DistributionScreens, PaymentCollectScreen, ReturnScreen}.kt

  Kinematic-iOS
  ├── Kinematic/Kinematic/Models/DistributionModels.swift
  ├── Kinematic/Kinematic/Services/OrderCache.swift
  ├── Kinematic/Kinematic/Services/DistributionAPI.swift
  ├── Kinematic/Kinematic/ViewModels/DistributionViewModel.swift
  ├── Kinematic/Kinematic/Views/Distribution/         (7 views)
  └── Kinematic/Kinematic/KinematicApp.swift          SecondaryRoute additions
```